

```

# =====
# 3.1 Simple Reflex Agent 5-9
# =====
class SimpleReflexEnvironment:
    def __init__(self, state='Dirty'):
        self.state = state

    def get_percept(self):
        return self.state

    def clean_room(self):
        self.state = 'Clean'

class SimpleReflexAgent:
    def __init__(self):
        pass

    def act(self, percept):
        if percept == 'Dirty':
            return 'Clean the room'
        else:
            return 'Room is already clean'

def run_simple_reflex_agent(agent, environment, steps):
    for step in range(steps):
        percept = environment.get_percept()
        action = agent.act(percept)
        print(f"Step {step + 1}: Percept {percept}, Action {action}")
        if percept == 'Dirty':
            environment.clean_room()

# =====
# 3.2 Model Based Agent 9-13
# =====
class ModelBasedEnvironment:
    def __init__(self, state='Dirty'):
        self.state = state

    def get_percept(self):
        return self.state

    def clean_room(self):
        self.state = 'Clean'

class ModelBasedAgent:

```

```

def __init__(self):
    self.model = {}

def update_model(self, percept):
    self.model['current'] = percept
    print(self.model)

def predict_action(self):
    if self.model['current'] == 'Dirty':
        return 'Clean the room'
    else:
        return 'Room is clean'

def act(self, percept):
    self.update_model(percept)
    return self.predict_action()

def run_model_based_agent(agent, environment, steps):
    for step in range(steps):
        percept = environment.get_percept()
        action = agent.act(percept)
        print(f"Step {step + 1}: Percept {percept}, Action {action}")
        if percept == 'Dirty':
            environment.clean_room()

# =====
# 3.3 Goal Based Agent 13-16
# =====
class GoalEnvironment:
    def __init__(self, state='Dirty'):
        self.state = state

    def get_percept(self):
        return self.state

    def clean_room(self):
        self.state = 'Clean'

class GoalBasedAgent:
    def __init__(self):
        self.goal = 'Clean'

    def formulate_goal(self, percept):
        if percept == 'Dirty':
            self.goal = 'Clean'

```

```

        else:
            self.goal = 'No action needed'

def act(self, percept):
    self.formulate_goal(percept)
    if self.goal == 'Clean':
        return 'Clean the room'
    else:
        return 'Room is clean'

def run_goal_based_agent(agent, environment, steps):
    for step in range(steps):
        percept = environment.get_percept()
        action = agent.act(percept)
        print(f"Step {step + 1}: Percept {percept}, Action {action}")
        if percept == 'Dirty':
            environment.clean_room()

# =====
# 3.4 Utility Based Agent
# =====

class UtilityEnvironment:
    def __init__(self, state='Dirty'):
        self.state = state

    def get_percept(self):
        return self.state

    def clean_room(self):
        self.state = 'Clean'

class UtilityBasedAgent:
    def __init__(self):
        self.utility = {'Dirty': -10, 'Clean': 10}

    def calculate_utility(self, percept):
        return self.utility[percept]

    def select_action(self, percept):
        if percept == 'Dirty':
            return 'Clean the room'
        else:
            return 'No action needed'

    def act(self, percept):

```

```

        action = self.select_action(percept)
        return action

def run_utility_based_agent(agent, environment, steps):
    total_utility = 0
    for step in range(steps):
        percept = environment.get_percept()
        action = agent.act(percept)
        utility = agent.calculate_utility(percept)
        print(f"Step {step + 1}: Percept {percept}, Action {action}, Utility
{utility}")
        total_utility += utility
        if percept == 'Dirty':
            environment.clean_room()
        print("Total Utility:", total_utility)

# =====
# 3.1 Breadth First Search 4
# =====
def bfs(tree, start, goal):
    visited = []
    queue = []
    visited.append(start)
    queue.append(start)
    while queue:
        node = queue.pop(0)
        print(node, end="")
        if node == goal:
            print("\nGoal found!")
            break
        for neighbour in tree.get(node, []):
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

# BFS Goal-Based Agent
class BFSGoalBasedAgent:
    def __init__(self, goal):
        self.goal = goal

    def formulate_goal(self, percept):
        if percept == self.goal:
            return "Goal reached"
        return "Searching"

```

```

def bfs_search(self, graph, start, goal):
    visited = []
    queue = []
    visited.append(start)
    queue.append(start)
    while queue:
        node = queue.pop(0)
        print(f"Visiting: {node}")
        if node == goal:
            return f"Goal {goal} found!"
        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)
    return "Goal not found"

def act(self, percept, graph):
    goal_status = self.formulate_goal(percept)
    if goal_status == "Goal reached":
        return f"Goal {self.goal} found!"
    else:
        return self.bfs_search(graph, percept, self.goal)

class SearchEnvironment:
    def __init__(self, graph):
        self.graph = graph

    def get_percept(self, node):
        return node

def run_search_agent(agent, environment, start_node):
    percept = environment.get_percept(start_node)
    action = agent.act(percept, environment.graph)
    print(action)

# =====
# 3.2 Depth First Search 10
# =====
def dfs(graph, start, goal):
    visited = []
    stack = []
    visited.append(start)
    stack.append(start)
    while stack:
        node = stack.pop()

```

```

    print(node, end="")
    if node == goal:
        print("\nGoal found!")
        break
    for neighbour in reversed(graph.get(node, [])):
        if neighbour not in visited:
            visited.append(neighbour)
            stack.append(neighbour)

# DFS Goal-Based Agent
class DFSGoalBasedAgent:
    def __init__(self, goal):
        self.goal = goal

    def formulate_goal(self, percept):
        if percept == self.goal:
            return "Goal reached"
        return "Searching"

    def dfs_search(self, graph, start, goal):
        visited = []
        stack = []
        visited.append(start)
        stack.append(start)
        while stack:
            node = stack.pop()
            print(f"Visiting: {node}")
            if node == goal:
                return f"Goal {goal} found!"
            for neighbour in reversed(graph.get(node, [])):
                if neighbour not in visited:
                    visited.append(neighbour)
                    stack.append(neighbour)
        return "Goal not found"

    def act(self, percept, graph):
        goal_status = self.formulate_goal(percept)
        if goal_status == "Goal reached":
            return f"Goal {self.goal} found!"
        else:
            return self.dfs_search(graph, percept, self.goal)

# =====
# 3.3 Depth Limited Search 14
# =====

```

```

def dls(graph, start, goal, depth_limit):
    visited = []
    def dfs_limit(node, depth):
        if depth > depth_limit:
            return None
        visited.append(node)
        if node == goal:
            return visited
        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                path = dfs_limit(neighbor, depth + 1)
                if path:
                    return path
        visited.pop()
        return None

    result = dfs_limit(start, 0)
    if result:
        print(f"Goal found with DLS. Path: {result}")
    else:
        print("Goal not found in this depth")

# DLS Goal-Based Agent (Lab 4 Task 1)
class DLSTaskAgent:
    def __init__(self, goal, depth_limit):
        self.goal = goal
        self.depth_limit = depth_limit

    def formulate_goal(self, percept):
        if percept == self.goal:
            return "Goal reached"
        return "Searching"

    def act(self, percept, graph):
        goal_status = self.formulate_goal(percept)
        if goal_status == "Goal reached":
            return f"Goal {self.goal} found immediately!"
        else:
            visited = []
            def dfs_agent(node, depth):
                if depth > self.depth_limit: return False
                visited.append(node)
                if node == self.goal: return True
                for neighbor in graph.get(node, []):
                    if neighbor not in visited:

```

```

        if dfs_agent(neighbor, depth + 1): return True
    visited.pop()
    return False

    success = dfs_agent(percept, 0)
    if success:
        return f"Agent successfully navigated to {self.goal} within depth
limit!"
    else:
        return "Agent hit depth limit before finding goal."

```

```

# =====
# 3.4 Iterative Deepening Depth First Search 16
# =====

```

```

def ids_dls_helper(node, goal, depth, path, tree):
    if depth == 0:
        return False
    if node == goal:
        path.append(node)
        return True
    if node not in tree:
        return False
    for child in tree.get(node, []):
        if ids_dls_helper(child, goal, depth - 1, path, tree):
            path.append(node)
            return True
    return False

def iterative_deepening(start, goal, max_depth, tree):
    for depth in range(max_depth + 1):
        print(f"Depth: {depth}")
        path = []
        if ids_dls_helper(start, goal, depth, path, tree):
            print("\nPath to goal:", " -> ".join(reversed(path)))
            return
    print("Goal not found within depth limit.")

```

```

# =====
# 3.5 Uniform Cost Search
# =====

```

```

def ucs(graph, start, goal):
    frontier = [(start, 0)]
    visited = set()
    cost_so_far = {start: 0}
    came_from = {start: None}

```

```

while frontier:
    frontier.sort(key=lambda x: x[1])
    current_node, current_cost = frontier.pop(0)

    if current_node in visited:
        continue
    visited.add(current_node)

    if current_node == goal:
        path = []
        while current_node is not None:
            path.append(current_node)
            current_node = came_from[current_node]
        path.reverse()
        print(f"Goal found with UCS. Path: {path}, Total Cost: {current_cost}")
        return

    for neighbor, cost in graph.get(current_node, {}).items():
        new_cost = current_cost + cost
        if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
            cost_so_far[neighbor] = new_cost
            came_from[neighbor] = current_node
            frontier.append((neighbor, new_cost))
    print("Goal not found")

# UCS Utility-Based Agent (Lab 4 Task 1)
class UCSUtilityBasedAgent:
    def __init__(self, goal):
        self.goal = goal

    def act(self, percept, graph):
        # Uses UCS logic to find minimum cost path to goal
        frontier = [(percept, 0)]
        visited = set()
        cost_so_far = {percept: 0}
        came_from = {percept: None}

        while frontier:
            frontier.sort(key=lambda x: x[1])
            current_node, current_cost = frontier.pop(0)

            if current_node in visited: continue
            visited.add(current_node)

```

```

        if current_node == self.goal:
            path = []
            while current_node is not None:
                path.append(current_node)
                current_node = came_from[current_node]
            path.reverse()
            return f"Utility Agent found optimal path to {self.goal}: {path}"
    with Cost: {current_cost}"

    for neighbor, cost in graph.get(current_node, {}).items():
        new_cost = current_cost + cost
        if neighbor not in cost_so_far or new_cost <
cost_so_far[neighbor]:
            cost_so_far[neighbor] = new_cost
            came_from[neighbor] = current_node
            frontier.append((neighbor, new_cost))

    return "Goal is unreachable."

# =====
# 2.1 Best First search 4
# =====
from queue import PriorityQueue

def best_first_search(graph, start, goal):
    visited = set()
    pq = PriorityQueue()
    pq.put((0, start))

    while not pq.empty():
        cost, node = pq.get()
        if node not in visited:
            print(node, end=' ')
            visited.add(node)
            if node == goal:
                print("\nGoal reached!")
                return True
            for neighbor, weight in graph.get(node, []):
                if neighbor not in visited:
                    pq.put((weight, neighbor))
    print("\nGoal not reachable!")
    return False

# =====

```

2.2 Greedy Best First Search 6

=====

```
def greedy_bfs(graph, heuristic, start, goal):
    frontier = [(start, heuristic[start])]
    visited = set()
    came_from = {start: None}

    while frontier:
        frontier.sort(key=lambda x: x[1])
        current_node, _ = frontier.pop(0)

        if current_node in visited: continue

        print(current_node, end=" ")
        visited.add(current_node)

        if current_node == goal:
            path = []
            while current_node is not None:
                path.append(current_node)
                current_node = came_from[current_node]
            path.reverse()
            print(f"\nGoal found with GBFS. Path: {path}")
            return

        for neighbor in graph.get(current_node, []):
            if neighbor not in visited:
                came_from[neighbor] = current_node
                frontier.append((neighbor, heuristic[neighbor]))

    print("\nGoal not found")
```

=====

2.3 A* Search

=====

```
def a_star(graph, heuristic, start, goal):
    frontier = [(start, heuristic[start])]
    visited = set()
    g_costs = {start: 0}
    came_from = {start: None}

    while frontier:
        frontier.sort(key=lambda x: x[1])
        current_node, current_f = frontier.pop(0)

        if current_node in visited: continue
```

```

print(current_node, end=" ")
visited.add(current_node)

if current_node == goal:
    path = []
    while current_node is not None:
        path.append(current_node)
        current_node = came_from[current_node]
    path.reverse()
    print(f"\nGoal found with A*. Path: {path}")
    return

for neighbor, cost in graph.get(current_node, {}).items():
    new_g_cost = g_costs[current_node] + cost
    f_cost = new_g_cost + heuristic[neighbor]

    if neighbor not in g_costs or new_g_cost < g_costs[neighbor]:
        g_costs[neighbor] = new_g_cost
        came_from[neighbor] = current_node
        frontier.append((neighbor, f_cost))

print("\nGoal not found")

# =====
# 2.1.1 Beam Search 4
# =====
import heapq

def beam_search(graph, start, goal, beam_width=2):
    beam = [(0, [start])]
    while beam:
        candidates = []
        for cost, path in beam:
            current_node = path[-1]
            if current_node == goal:
                return path, cost
            for neighbor, edge_cost in graph.get(current_node, []):
                new_cost = cost + edge_cost
                new_path = path + [neighbor]
                candidates.append((new_cost, new_path))
        beam = heapq.nsmallest(beam_width, candidates, key=lambda x: x[0])
    return None, float('inf')

# =====
# 2.1.2 Hill Climbing 6

```

```

# =====
import random

def calculate_conflicts(state):
    conflicts = 0
    n = len(state)
    for i in range(n):
        for j in range(i+1, n):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1
    return conflicts

def get_neighbors(state):
    neighbors = []
    n = len(state)
    for row in range(n):
        for col in range(n):
            if col != state[row]:
                new_state = list(state)
                new_state[row] = col
                neighbors.append(new_state)
    return neighbors

def simple_hill_climbing(n):
    current_state = [random.randint(0, n-1) for _ in range(n)]
    current_conflicts = calculate_conflicts(current_state)

    while True:
        neighbors = get_neighbors(current_state)
        best_neighbor = None
        best_conflicts = current_conflicts

        for neighbor in neighbors:
            conflicts = calculate_conflicts(neighbor)
            if conflicts < best_conflicts:
                best_neighbor = neighbor
                best_conflicts = conflicts

        if best_conflicts >= current_conflicts:
            break # Reached local maxima or plateau

        current_state = best_neighbor
        current_conflicts = best_conflicts

    return current_state, current_conflicts

```

```

# =====
# 2.1.3 Genetic Algorithm
# =====
import random

def calculate_fitness_ga(individual, n):
    non_attacking_pairs = 0
    for i in range(n):
        for j in range(i+1, n):
            if individual[i] != individual[j] and abs(individual[i] -
individual[j]) != abs(i - j):
                non_attacking_pairs += 1
    return non_attacking_pairs

def create_random_individual(n):
    return random.sample(range(n), n)

def select_parents_ga(population, fitness_scores):
    sorted_population = [board for _, board in sorted(zip(fitness_scores,
population), reverse=True)]
    return sorted_population[:len(population)//2]

def crossover_ga(parent1, parent2, n):
    point = random.randint(1, n-2)
    child = parent1[:point] + parent2[point:]
    missing = set(range(n)) - set(child)
    for i in range(len(child)):
        if child.count(child[i]) > 1:
            child[i] = missing.pop()
    return child

def mutate_ga(individual, n, mutation_rate=0.1):
    if random.random() < mutation_rate:
        idx1, idx2 = random.sample(range(n), 2)
        individual[idx1], individual[idx2] = individual[idx2], individual[idx1]
    return individual

def genetic_algorithm(n, population_size=10, max_generations=100):
    population = [create_random_individual(n) for _ in range(population_size)]
    best_overall = None
    best_fitness = -1

    for generation in range(max_generations):
        fitness_scores = [calculate_fitness_ga(ind, n) for ind in population]

```

```

    current_best_fit = max(fitness_scores)
    if current_best_fit > best_fitness:
        best_fitness = current_best_fit
        best_overall = population[fitness_scores.index(current_best_fit)]

    parents = select_parents_ga(population, fitness_scores)
    new_population = []

    while len(new_population) < population_size:
        p1, p2 = random.sample(parents, 2)
        child = crossover_ga(p1, p2, n)
        child = mutate_ga(child, n)
        new_population.append(child)

    population = new_population

    return best_overall, best_fitness

# =====
# 2.1.3 Genetic Algorithm
# =====
import random

# -----
# EXAMPLE 1: Solving N-Queens Using Genetic Algorithm
# -----
print("--- GA Example 1: N-Queens Problem ---")

n_queens = 8 # Standard 8-Queens board
population_size_queens = 10
mutation_rate_queens = 0.1

# Step 1: Define the Problem and Fitness Function
def calculate_fitness_queens(individual):
    non_attacking_pairs = 0
    total_pairs = n_queens * (n_queens - 1) // 2 # Maximum possible non-attacking
pairs

    # Check for conflicts
    for i in range(n_queens):
        for j in range(i + 1, n_queens):
            # No same column or diagonal conflict
            if individual[i] != individual[j] and abs(individual[i] -
individual[j]) != abs(i - j):

```

```

        non_attacking_pairs += 1
    return non_attacking_pairs

# Step 2: Initialize the Population
def create_random_individual_queens():
    return random.sample(range(n_queens), n_queens) # Ensure unique column
positions

# Step 4: Selection
def select_parents_queens(population, fitness_scores):
    # Select top 50% of the population
    sorted_population = [board for _, board in sorted(zip(fitness_scores,
population), reverse=True)]
    return sorted_population[:len(population)//2]

# Step 5: Crossover function: single-point crossover with unique column
def crossover_queens(parent1, parent2):
    point = random.randint(1, n_queens - 2) # Choose a crossover point
    child = parent1[:point] + parent2[point:]

    # Ensure unique column positions
    missing = set(range(n_queens)) - set(child)
    for i in range(len(child)):
        if child.count(child[i]) > 1:
            child[i] = missing.pop()
    return child

# Step 6: Randomly swap two locations in a route (Mutation)
def mutate_queens(individual):
    idx1, idx2 = random.sample(range(n_queens), 2)
    individual[idx1], individual[idx2] = individual[idx2], individual[idx1]
    return individual

# Step 7: GA Main Function / Check Convergence Criteria
def genetic_algorithm_queens(max_generations=100):
    population = [create_random_individual_queens() for _ in
range(population_size_queens)]
    best_fitness = 0
    best_solution = None

    for generation in range(max_generations):
        # Step 3: Evaluate Fitness
        fitness_scores = [calculate_fitness_queens(ind) for ind in population]

        current_best = max(fitness_scores)

```

```

    if current_best > best_fitness:
        best_fitness = current_best
        best_solution = population[fitness_scores.index(current_best)]

    parents = select_parents_queens(population, fitness_scores)

    # Create new population using crossover and mutation
    new_population = []
    while len(new_population) < population_size_queens:
        parent1, parent2 = random.sample(parents, 2)
        child = crossover_queens(parent1, parent2)

        if random.random() < mutation_rate_queens:
            child = mutate_queens(child)

        new_population.append(child)

    population = new_population

    print(f"Best N-Queens Solution Found: {best_solution}")
    print(f"Non-Attacking Pairs: {best_fitness}")

# Run N-Queens GA
genetic_algorithm_queens()

# -----
# EXAMPLE 2: Duty Scheduling Problem Using Genetic Algorithm
# -----
print("\n--- GA Example 2: Duty Scheduling Problem ---")

# Configuration
num_staff = 5 # Number of employees
num_shifts = 21 # 7 days * 3 shifts per day
max_shifts_per_staff = 7
required_staff_per_shift = 2
population_size_sched = 10
mutation_rate_sched = 0.1
max_generations_sched = 1000

# Step 1: Fitness function (lower is better, we track penalty)
def evaluate_fitness_sched(schedule):
    penalty = 0
    # Check shift coverage

```

```

    for shift in range(num_shifts):
        assigned_count = sum(schedule[staff][shift] for staff in
range(num_staff))
        if assigned_count < required_staff_per_shift:
            penalty += (required_staff_per_shift - assigned_count) * 10
# Note: Checking for consecutive shifts adds a penalty of 5 per occurrence
        for staff in range(num_staff):
            for shift in range(num_shifts - 1):
                if schedule[staff][shift] == 1 and schedule[staff][shift+1] == 1:
                    penalty += 5
    return penalty

# Step 2: Create a random schedule
def create_random_schedule():
    schedule = [[0] * num_shifts for _ in range(num_staff)]
    for staff in range(num_staff):
        assigned_shifts = random.sample(range(num_shifts), random.randint(3,
max_shifts_per_staff))
        for shift in assigned_shifts:
            schedule[staff][shift] = 1
    return schedule

# Step 3: Selection (Top 50%)
def select_parents_sched(population, fitness_scores):
    # Sort ascending because lower penalty is better!
    sorted_population = [x for _, x in sorted(zip(fitness_scores, population))]
    return sorted_population[:len(population) // 2]

# Step 4: Crossover (Single point crossover)
def crossover_sched(parent1, parent2):
    point = random.randint(0, num_shifts - 1)
    child = [parent1[i][:point] + parent2[i][point:] for i in range(num_staff)]
    return child

# Step 5: Mutation (Swap shifts for one staff)
def mutate_sched(schedule):
    staff = random.randint(0, num_staff - 1)
    shift1, shift2 = random.sample(range(num_shifts), 2)
    schedule[staff][shift1], schedule[staff][shift2] = schedule[staff][shift2],
schedule[staff][shift1]
    return schedule

# Step 6: Convergence Criteria & GA Loop
def genetic_algorithm_scheduling():
    # Initial population

```

```

population = [create_random_schedule() for _ in range(population_size_sched)]

# Genetic Algorithm loop
for generation in range(max_generations_sched):
    fitness_scores = [evaluate_fitness_sched(schedule) for schedule in
population]
    best_fitness = min(fitness_scores)

    # To avoid spamming console, we only print every 200 generations
    if generation % 200 == 0 or generation == max_generations_sched - 1:
        print(f"Generation {generation + 1}, Best Fitness (Penalty):
{best_fitness}")

    parents = select_parents_sched(population, fitness_scores)

    new_population = []
    while len(new_population) < population_size_sched:
        parent1, parent2 = random.sample(parents, 2)
        child = crossover_sched(parent1, parent2)

        if random.random() < mutation_rate_sched:
            child = mutate_sched(child)

        new_population.append(child)

    population = new_population

# Fetch Best schedule at the end
final_fitness_scores = [evaluate_fitness_sched(schedule) for schedule in
population]
best_schedule =
population[final_fitness_scores.index(min(final_fitness_scores))]

print("\nBest Schedule (Staff x Shifts):")
for staff in range(num_staff):
    print(f"Staff {staff + 1}: {best_schedule[staff]}")

# Run Scheduling GA
genetic_algorithm_scheduling()

# =====
# LAB TASK 3: Minimize Assignment Cost (Genetic Algorithm)
# =====
import random

```

The 10x5 Cost Matrix provided in the lab manual

```
cost_matrix = [  
    [4, 6, 8, 7, 5],  
    [7, 5, 6, 8, 4],  
    [6, 4, 7, 5, 8],  
    [5, 8, 6, 4, 7],  
    [8, 6, 5, 7, 4],  
    [7, 4, 8, 6, 5],  
    [6, 7, 4, 5, 8],  
    [5, 6, 7, 8, 4],  
    [4, 7, 5, 6, 8],  
    [8, 5, 6, 4, 7]  
]
```

```
num_tasks = 10
```

```
num_machines = 5
```

```
max_generations = 100
```

```
mutation_rate = 0.1
```

```
def create_random_chromosome():
```

```
    # A list of 10 random integers between 0 and 4 representing machine  
    assignments
```

```
    return [random.randint(0, num_machines - 1) for _ in range(num_tasks)]
```

```
def get_total_cost(chromosome):
```

```
    total_cost = 0
```

```
    for task_index in range(num_tasks):
```

```
        machine_index = chromosome[task_index]
```

```
        total_cost += cost_matrix[task_index][machine_index]
```

```
    return total_cost
```

```
def calculate_fitness(chromosome):
```

```
    # Fitness = 1 / total cost
```

```
    return 1.0 / get_total_cost(chromosome)
```

```
def select_parents(population, fitness_scores):
```

```
    # Select Top 50%
```

```
    sorted_population = [chromosome for score, chromosome in  
sorted(zip(fitness_scores, population), reverse=True)]
```

```
    return sorted_population[:len(population) // 2]
```

```
def crossover(parent1, parent2):
```

```
    # Single-point crossover
```

```
    point = random.randint(1, num_tasks - 1)
```

```
    child = parent1[:point] + parent2[point:]
```

```

    return child

def mutate(chromosome):
    # Randomly change the machine assignment for one random task
    task_to_mutate = random.randint(0, num_tasks - 1)
    new_machine = random.randint(0, num_machines - 1)
    chromosome[task_to_mutate] = new_machine
    return chromosome

def run_task3_genetic_algorithm(pop_size):
    print(f"\n--- Running Task 3 GA with Population Size: {pop_size} ---")

    population = [create_random_chromosome() for _ in range(pop_size)]

    overall_best_chromosome = None
    overall_best_cost = float('inf')
    overall_best_fitness = 0
    generation_found = 0

    for generation in range(max_generations):
        fitness_scores = [calculate_fitness(ind) for ind in population]

        current_best_fitness = max(fitness_scores)
        if current_best_fitness > overall_best_fitness:
            best_index = fitness_scores.index(current_best_fitness)
            overall_best_fitness = current_best_fitness
            overall_best_chromosome = population[best_index]
            overall_best_cost = get_total_cost(overall_best_chromosome)
            generation_found = generation + 1

        parents = select_parents(population, fitness_scores)

        new_population = []
        while len(new_population) < pop_size:
            p1, p2 = random.sample(parents, 2)
            child = crossover(p1, p2)

            if random.random() < mutation_rate:
                child = mutate(child)

            new_population.append(child)

        population = new_population

    print(f"Best Chromosome (Task->Machine): {overall_best_chromosome}")

```

```
print(f"Total Cost: {overall_best_cost}")
print(f"Fitness Score: {overall_best_fitness:.5f}")
print(f"Found in Generation: {generation_found}")

# Run comparisons as requested by the lab task
run_task3_genetic_algorithm(10)
run_task3_genetic_algorithm(30)
```